

HelixSkills

HelixSkills

CLI AIエージェント向けの統治された、憲法に基づくスキルシステム

概要

HelixSkillsは、CLI AIエージェント向けのスキルシステムであり、Helix Constitutionをサブモジュールとして継承しているため、すべての普遍的な統治ルールが無条件に適用されます。このシステムは、インストール可能なエージェントスキル、MCPツールサーバー、Claude Codeプラグイン、そして再利用可能なエンジンを、登録可能で文書化されたカタログとしてパッケージ化しています。

短い説明

HelixSkillsは、CLI AIエージェント向けのスキルシステムです。Helix Constitutionをサブモジュールとして組み込んでいるため、すべての普遍的なルールが適用され、その後に登録可能なスキル（アクションプレフィックス、メディアバリデーター、マルチトラック、セッション同期、ワーク可能アイテムのライフサイクルなど）、2つのMCPツールサーバー、2つのClaude Codeプラグイン、そして再利用可能なエンジンを提供します。

長い説明

HelixSkills（リポジトリ名 skills、Apache-2.0ライセンス）は、CLI AIエージェント向けのスキルシステムであり、従来の順序を意図的に逆転させた設計から始まります。すなわち、機能よりも統治が優先されます。このシステムはHelix Constitutionをconstitution/サブモジュールとして継承しているため、constitution/CLAUDE.mdおよびconstitution/Constitution.mdに記載されたすべての普遍的なルールが無条件に適用されます。これは単なる慣習としてエージェントが遵守するものではなく、プロジェクトツリーに物理的に組み込まれたルールセットです。HelixSkillsを採用したエージェントは、憲法のルールをオプトアウトすることはできません。ルールはコードとともに移動します。

多くの「スキルフレームワーク」が抽象的な概念を扱うのに対し、HelixSkillsは具体的で登録可能なインベントリを提供し、それを指定してインストールすることができます。7つの憲法スキルはregister.shを通じてインストールされます: action-prefix-system、media-validator、multitrack、reporting-workable-items、scheduled-work-queue、session-sync、workable-item-lifecycle。これらは中級から上級レベルまでの幅広い機能をカバーし、規律あるアクション命名からメディア検証、さらには作業単位の完全なライフサイクル管理に至るまでを網羅しています。また、追加のドラフトスキル（Android概要、Java/Kotlin言語、Linux OS）もすでにインデックス化され、有効化を待機中です。2つのMCPツールサーバー（media-validator、scheduled-work）は、これらのスキルをModel Context Protocolを介してエージェントに提供し、2つのClaude Codeプラグイン（helix、scheduled-work）は同じ機能をエージェントのランタイムに直接組み込みます。つまり、1つのスキルセットが、エージェントが使用するインターフェースに応じて提供されるのです。

カタログの背後には、4つの深さ1の再利用可能なエンジンが存在します。continuum（実装済み）に加え、session_orchestrator、token_optimizer、clickup_sync（設計中）が含まれます。これらはスキルが共通の基盤を再構築することを防ぐ共有機構です。特にtoken_optimizerは、vasic-digitalエコシステムパッケージ（TOON、Embeddings、VectorDB、Normalize、conversation）およびHelixDevelopmentのLLMProviderに至る明示的な依存関係グラフを宣言しているため、そのリポジトリ間の接続は暗黙的ではなく監査可能です。全体を通じて、規律あるドキュメントが整備されています: スキルカタログ、自動生成されたスキルグラフインデックス、各リポジトリの詳細ページ、そしてまだ実装されていない部分を明示する「Gaps & Risks」レジスタです。システム全体は、GitHub、GitLab、GitFlic、GitVerseにミラーリングされており、耐障害性と地域アクセスを確保しています。

コンテンツ

なぜこれを構築したのか

CLIやAIのエージェントには、一貫性があり、統制され、再利用可能な機能が求められます。その場しのぎのスクリプトでルールを毎回作り直すのではなく、HelixSkillsはエージェントにパッケージ化され、登録可能なスキルセットを提供します。これは共有の憲章に基づいており、採用するすべてのエージェントやプロジェクトにおいて、振る舞いの一貫性と監査可能性が保たれます。

なぜゲームチェンジャーなのか

エージェントの機能を、規律ではなく「構造的に」ポータブルかつルール準拠にすることができます。すべてのスキルは、憲章のサブモジュールによって統制され、バージョン管理され、インストール可能な単位として提供されます。そのため、エージェントがスキルを登録した瞬間に、標準的なルールセットを継承し、逸脱の余地はありません。これにより、これま

で実現不可能だったことが可能になります。あるエージェントやプロジェクトから別のエージェントやプロジェクトへ機能を移行しても、同じガバナンスに最初から縛られていることが保証されます。しかも、その機能は、各自がルールを再発明する場当たりのな接着スクリプトの山ではなく、標準的なインターフェース（MCPサーバーやClaude Codeプラグイン）を通じて提供されます。

革新的なポイント

- サブモジュールとしての**Constitution**：普遍的なガバナンスルールはコピーされるのではなく継承され、ツリーにマウントされることで、すべての利用エージェントが同じ標準ルールセットに縛られます。更新は単一の情報源から流れるため、陳腐化したコピーが十数個も生まれることはありません。
- 自己登録型スキルユニット：各スキルはregister.shによって登録され、インストール時に自動的に組み込まれます。さらに、自動生成されるスキルグラフインデックスとリポジトリごとの詳細ドキュメントにより、カタログは常に実際にインストールされている内容と同期し、発見可能性が保たれます。
- マルチサーフェス対応：同一のスキルセットがMCPツールサーバーとClaude Codeプラグインの両方を通じてエージェントに提供されます。一度書けば、エージェントが使用するどのランタイムにも対応可能です。
- 再利用可能な深さ1のエンジン：「continuum」「token_optimizer」「session_orchestrator」「clickup_sync」などのエンジンがエコシステム全体で共有され、リポジトリ間の依存関係が明示的かつ監査可能な形で宣言されます。これにより、隠れた結合が排除されます。

最大の技術的課題とその解決策

- 多数のスキルとエージェントにわたって振る舞いの一貫性とルール準拠を維持すること — スキルごとにガバナンスを再実装すると、時間とともに逸脱が生じます。この問題は、Helix Constitutionをサブモジュールとしてマウントすることで解決しました。これにより、constitution/CLAUDE.mdやconstitution/Constitution.mdのルールが無条件に適用され、単一のアップストリームから更新されるため、コピーされて陳腐化することはありません。
- 増え続けるスキルセットをインストール可能かつ発見可能にすること — カatalogがあっても、その中身を誰も見つけられなければ意味がありません。この課題は、スキルごとのregister.shによる登録と、インストール時の自動組み込み、さらに自動生成されるINDEXスキルグラフとリポジトリごとの詳細ドキュメントによって解決しました。これにより、発見可能性が現実在即して自動的に追従します。
- 異なるランタイムを持つエージェントに対応すること — 同じ機能をホストごとに再構築する必要はありません。この問題は、MCPツールサーバー定義（constitution/mcp/配下）とClaude Codeプラグイン（constitution/plugins/配下）の両方で同一のスキルセットをパッ

ケース化することで解決しました。これにより、単一の実装が複数のインターフェースを通じて提供されます。

テックスタック

- **Shell** (主言語) — エージェントが稼働するあらゆる環境で、事前のランタイムブートストラップなしにインストールや登録ツールを実行できるよう選定。register.shやinstall_upstreamsを駆動し、導入時の依存関係を排除したポータブルな設計を実現。
- **Git**サブモジュール — ガバナンスを重複なく継承するために採用。Helix Constitutionはconstitution/にライブラリファレンスとしてマウントされており、ルールを更新はコピー&ペーストではなく、一つのポインタを通じて伝播。
- **Model Context Protocol (MCP)** — エージェント向けの標準的なランタイム非依存ツールインターフェースとして選定。constitution/mcp/配下に二つのMCPサーバー (media-validator、scheduled-work) を定義し、スキルを呼び出し可能なツールとして公開。
- **Claude**コードプラグイン — エージェントランタイムにスキルをネイティブに組み込むために採用。一切の接着コードなしで実装可能。二つのプラグイン (helix、scheduled-work) はconstitution/plugins/配下に配置され、異なるホスト向けにMCPのインターフェースをミラーリング。
- 再利用可能エンジン (**continuum、token_optimizer、session_orchestrator、clickup_sync**) — 個々のスキルから共通機能を切り出し、プロジェクト横断での再利用を可能にするために採用。例えばtoken_optimizerは、vasic-digitalのパッケージ群 (TOON、Embeddings、VectorDB、Normalize、conversation) やHelixDevelopmentのLLMProviderと、コードの重複ではなく宣言的な依存関係で連携。
- マルチホスト**Git**ミラーリング (**GitHub、GitLab、GitFlic、GitVerse**) — 単一ホストの障害や地域ブロックによるアクセス遮断を防ぐために採用。同一リポジトリを四つのフォージで常時同期し、耐障害性とアクセス性を確保。

ステータスと正直な注意点

- ステータス: ベータ版。七つの憲法スキル、二つのMCPサーバー、二つのプラグインはリリース済み。ドラフトスキルはインデックス化されており、有効化待ち。四つの深度1エンジン (session_orchestrator、token_optimizer、clickup_sync) のうち三つはまだ設計段階。
- READMEではプロジェクトをhelix_skillsと表記しているが、正式なGitHubのパスはHelixDevelopment/skills。READMEに記載された「追跡対象の発見数」は自己申告による数値。

優先度区分: Helix-プライマリ。